

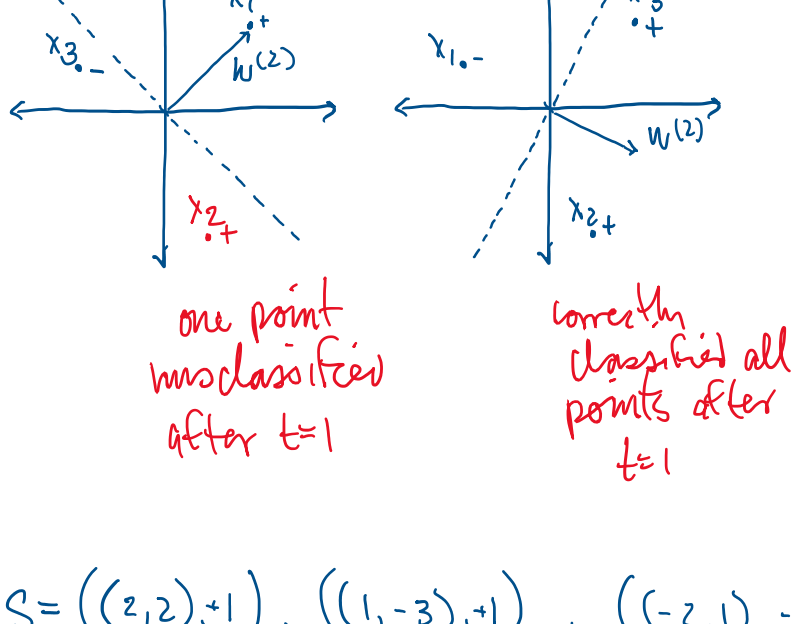
HW 2: Theory

Thursday, September 12, 2024

9:46 PM

1. [10 points] Consider running the Perceptron algorithm on a training set S arranged in a certain order. Now suppose we run it with the same initial weights and on the same training set but in a different order, S' . Does Perceptron make the same number of mistakes? Does it end up with the same final weights? If so, prove it. If not, give a counterexample, i.e. an S and S' where order matters.

Intuitively, the order of examples affect when updates are made and the classifier $\vec{w}$ may classify points differently depending on the order. I'll try geometrically to find a counterexample in the homogeneous case.



$$S = ((2, 2), +1), ((1, -3), +1), ((-2, 1), -1)$$

$$w^{(1)} = (0, 0)$$

$$t=1: \quad i=1: \quad 1 \cdot \langle w^{(1)}, x_1 \rangle = 1 \cdot 0 \leq 0$$

$$w^{(2)} = (0, 0) + 1 \cdot (2, 2) = (2, 2)$$

$$i=2: \quad 1 \cdot \langle w^{(2)}, x_2 \rangle = 1 \cdot \langle (2, 2), (1, -3) \rangle \leq 0$$

mistake!

$$S' = ((-2, 1), -1), ((1, -3), +1), ((2, 2), +1)$$

$$w^{(1)} = (0, 0)$$

$$t=1: \quad i=1: \quad (-1) \cdot \langle w^{(1)}, x_1 \rangle = -1 \cdot 0 \leq 0$$

$$w^{(2)} = (0, 0) + (-1) \cdot (-2, 1) = (2, -1)$$

all points correctly classified!
see diagram

Hence, S makes at least 2 mistakes while S' makes exactly 1 mistake.

2. [10 points] We have mainly focused on squared loss, but there are other interesting losses in machine learning. Consider the following loss function which we denote by $\phi(z) = \max(0, -z)$. Let S be a training set $(x^1, y^1), \dots, (x^m, y^m)$ where each $x^i \in \mathbb{R}^n$ and $y^i \in \{-1, 1\}$. Consider running stochastic gradient descent (SGD) to find a weight vector w that minimizes $\frac{1}{m} \sum_{i=1}^m \phi(y^i \cdot w^T x^i)$. Explain the explicit relationship between this algorithm and the Perceptron algorithm. Recall that for SGD, the update rule when the i^{th} example is picked at random is

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla \phi(y^i w^T x^i).$$

Note: You do not need to be overly concerned about the discontinuity at $\phi(0)$, so you can ignore this when calculating the gradient for this problem.

$$\phi(z) = \max(0, -z)$$

$$S = (x^1, y^1), \dots, (x^m, y^m), \quad x^i \in \mathbb{R}^n, \quad y^i \in \{-1, 1\}$$

use SGD to minimize $\frac{1}{m} \sum_{i=1}^m \phi(y^i \cdot w^T x^i)$

1. calc. $\nabla_w \phi(y^i w^T x^i)$

$$\nabla_w \max\{0, -y^i w^T x^i\}$$

$$y^i w^T x^i < 0: \quad \phi(y^i w^T x^i) = y^i w^T x^i$$

$$\boxed{\nabla_w y^i w^T x^i = y^i x^i}$$

2. At each iteration, for a random i :

$$\text{for } y^i w^T x^i < 0: \quad w_{\text{new}} = w_{\text{old}} - \eta \nabla \phi(y^i w^T x^i)$$

$$= w_{\text{old}} - \eta y^i x^i$$

for $y^i w^T x^i \geq 0$: we have the discontinuity we were told not to worry about.

$$\text{assume } \nabla \phi(y^i w^T x^i) = 0$$

$$w_{\text{new}} = w_{\text{old}}$$

3. With this loss, SGD is essentially performing on-line Perceptron w/ a random point sampled at each iteration, with an additional learning rate parameter. There is no penalty incurred when $y^i w^T x^i > 0$.

3. [6 points] Here we will give an illustrative example of a weak learner for a simple concept class. Let the domain be the real line, $\mathbb{R}$, and let $\mathcal{C}$ refer to the concept class of "3-piece classifiers", which are functions of the following form: for $\theta_1 < \theta_2$ and $b \in \{-1, 1\}$, $h_{\theta_1, \theta_2, b}(x)$ is b if $x \in [\theta_1, \theta_2]$ and $-b$ otherwise. In other words, they take a certain Boolean value inside a certain interval and the opposite value everywhere else. For example, $h_{10, 20, 1}(x)$ would be $+1$ on $[10, 20]$, and -1 everywhere else. Let $\mathcal{H}$ refer to the simpler class of "decision stumps", i.e. functions $h_{\theta, b}$ such that $h(x)$ is b for all $x \leq \theta$ and $-b$ otherwise.

- (a) Show formally that for any distribution on $\mathbb{R}$ (assume finite support, for simplicity; i.e., assume the distribution is bounded within $[-B, B]$ for some large B) and any unknown labeling function $c \in \mathcal{C}$ that is a 3-piece classifier, there exists a decision stump $h \in \mathcal{H}$ that has error at most $1/3$, i.e. $\mathbb{P}[h(x) \neq c(x)] \leq 1/3$.

$\mathcal{C}$: class of "3-piece classifiers"

$$\text{for } \theta_1 < \theta_2 \text{ and } b \in \{-1, 1\}, \quad h_{\theta_1, \theta_2, b}(x) = \begin{cases} b, & x \in [\theta_1, \theta_2] \\ -b & \text{otherwise} \end{cases}$$

$$\text{example: } h_{10, 20, 1}(x) = \begin{cases} 1, & x \in [10, 20] \\ -1 & \text{otherwise} \end{cases}$$

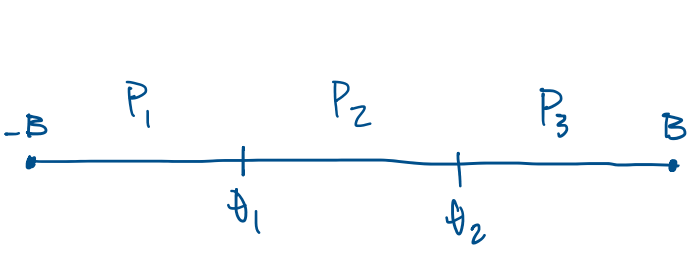
$\mathcal{H}$: class of "decision stumps"

$$h(x) = \begin{cases} b, & x \leq \theta \\ -b & \text{otherwise} \end{cases} \quad \text{where } b \in \{-1, 1\}?$$

(a) let D be arbitrary distr. on $\mathbb{R}$ bounded $[-B, B]$ for large B

for any $c \in \mathcal{C}$ (3-piece classifier)

show there exists $h \in \mathcal{H}$ (dec. stump) w/ error at most $1/3$, i.e. $\mathbb{P}[h(x) \neq c(x)] \leq 1/3$



$$\text{let } \mathbb{P}(X \in [-B, \theta_1]) = P_1$$

$$\mathbb{P}(X \in (\theta_1, \theta_2)) = P_2$$

$$\mathbb{P}(X \in (\theta_2, B]) = P_3$$

$$P_1 + P_2 + P_3 = 1$$

$$\text{choose } c \in \mathcal{C} = h_{\theta_1, \theta_2, b}(x) = \begin{cases} b, & x \in [\theta_1, \theta_2] \\ -b & \text{otherwise} \end{cases}$$

$$\text{consider } h_{\theta_1, -b}(x) = \begin{cases} -b, & x \leq \theta_1 \\ b, & x > \theta_1, \end{cases} \quad \epsilon = P_3$$

$$h_{\theta_2, b}(x) = \begin{cases} b, & x \leq \theta_2 \\ -b, & x > \theta_2, \end{cases} \quad \epsilon = P_1$$

$$h_{B, -b}(x) = -b, \quad x \leq B, \quad \epsilon = P_2$$

By triangle inequality at least one of P_1, P_2, P_3 , must have prob. mass $\leq 1/3$. Therefore, we can

always select one of h above to ensure error $\leq 1/3$. Works for $c_{\theta_1, \theta_2, b}$ by setting $-b = b$

and using the same proof.

- (b) Describe a simple, efficient procedure for finding a decision stump that minimizes error with respect to a finite training set of size m . Such a procedure is called an empirical risk minimizer (ERM).

input: $(x_1, y_1), \dots, (x_m, y_m)$

initialize: sort x_i ascending

$$\text{set } \theta^{(1)} = x_1$$

for $j = 1, 2, \dots, m$:

$$\text{set } \theta^{(j)} = x_j$$

$$\text{calculate } \frac{1}{m} \sum_{i=1}^m \mathbb{1}[h(x_i) \neq c(x_i)] = \epsilon^{(j)}$$

set $\theta = x_j, j = \arg \min_j \epsilon^{(j)}$

runtime: $O(m \log m) + O(m^2) = O(m^2)$

sorting

iterate m times

calculate error at each step

- (c) Give a short intuitive explanation for why we should expect that we can easily pick m sufficiently large that the training error is a good approximation of the true error, i.e. why we can ensure generalization. (Your answer should relate to what we have gained in going from requiring a learner for $\mathcal{C}$ to requiring a learner for $\mathcal{H}$.) This lets us conclude that we can weakly learn $\mathcal{C}$ using $\mathcal{H}$.

We know that $\mathcal{H}$ (decision stump w/ one param) is

simpler than $\mathcal{C}$ (3-piece classifier w/ two params),

so it can be learned w/ fewer training examples.

Lower VC implies greater generalization, i.e. less

likely to overfit. Therefore we can confidently

weakly learn $\mathcal{C}$ with sufficient examples. Then we

can optionally use boosting to get desired error.

4. [10 points] Consider an iteration of the AdaBoost algorithm (using notation from the video lecture on Boosting) where we have obtained classifier h_t . Show that with respect to the distribution D_{t+1} generated for the next iteration, h_t has accuracy exactly $1/2$.

$$\epsilon = \text{error rate} \quad A = \text{accuracy} = 1 - \epsilon \quad \beta = \frac{\epsilon}{1 - \epsilon}$$

$$\text{concretely } \epsilon = \frac{1}{2} - \gamma \quad \beta = \frac{\frac{1}{2} - \gamma}{\frac{1}{2} + \gamma}$$

W_t = sum of weights before iteration $t+1$

weights after iteration $t+1$

$$W_{t+1, \text{correct}} = (\frac{1}{2} + \gamma) \cdot \beta \cdot W_t$$

$$W_{t+1, \text{incorrect}} = (\frac{1}{2} - \gamma) \cdot W_t$$

$$W_{t+1, \text{total}} = (\frac{1}{2} + \gamma) \cdot \beta \cdot W_t + (\frac{1}{2} - \gamma) \cdot W_t$$

$$= W_t \left((\frac{1}{2} + \gamma) \beta + \frac{1}{2} - \gamma \right)$$

$$= W_t \left((\frac{1}{2} - \gamma) + \frac{1}{2} - \gamma \right)$$

$$= W_t (2(\frac{1}{2} - \gamma))$$

h_t has already been fit, so it will

classify all points $\{x_1, x_2, \dots, x_m\}$ the

same. This question is just about showing

how adjusting the weight pushes the

accuracy of h_t down to $1/2$, emphasizing

the idea that we are reducing the

importance of those correctly classified

points in the next iteration.

accuracy of h_t on its training set was $\frac{1}{2} + \gamma$

If we reweight the correct points by β and

calculate accuracy, we get:

$$\frac{W_{t+1, \text{correct}}}{W_{t+1, \text{total}}} = \frac{(\frac{1}{2} + \gamma) \cdot \beta \cdot W_t}{W_t (2(\frac{1}{2} - \gamma))} = \frac{(\frac{1}{2} + \gamma) \cdot \left(\frac{\frac{1}{2} - \gamma}{\frac{1}{2} + \gamma} \right)}{2(\frac{1}{2} - \gamma)} = \frac{1}{2}$$